



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическая работа № 4/4 ч.

Разработка мобильных приложений

Уровень	(наименование дисциплины (модуля) в соответствии с учебным планом) бакалавриат
Форма обучения	(бакалавриат, магистратура, специалитет) очная
Направление(-я) подготовки	(очная, очно-заочная, заочная) 09.03.02 «Информационные системы и технологии»
Институт	(код(-ы) и наименование(-я)) кибербезопасности и цифровых технологий
Кафедра	(полное и краткое наименование) КБ-14 «Цифровые технологии обработки данных»
Используются в данной редакции с учебного года	(полное и краткое наименование кафедры, реализующей дисциплину (модуль)) 2024/25
Проверено и согласовано « ____ » ____ 20 ____ г.	(учебный год цифрами) (подпись директора Института/Филиала с расшифровкой)

Москва 2024 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1 ОСНОВНЫЕ СПОСОБЫ ОТОБРАЖЕНИЯ СПИСКОВ.....	3
1.1 ScrollView	4
1.2 ListView.....	6
1.3 RecyclerView	9
2 КОНТРОЛЬНОЕ ЗАДАНИЕ.....	16

ВВЕДЕНИЕ

Списки являются важной частью многих Android-приложений, так как они позволяют отображать большие объемы данных в компактном и удобном для пользователя формате. В качестве примера возможно привести список контактов, чатов, новостей или товаров в интернет-магазине. В Android для работы со списками существует несколько компонентов, каждый из которых предназначен для решения определенных задач и обладает своими преимуществами.

Если необходимо отобразить большое количество информации, которая не уместится на экране, требуется добавить возможность скроллинга контента. Для этого в Android используется компонент *ScrollView*, который позволяет прокручивать элементы интерфейса вертикально или горизонтально. Он прост в использовании, однако подходит лишь для небольших наборов данных и статичных интерфейсов, так как он не предназначен для динамической подгрузки элементов и может приводить к проблемам с производительностью, поэтому его следует использовать только для отображения небольшого объема данных.

При необходимости отобразить более объемные списки данных Android SDK предлагает более специализированные компоненты: *ListView* и *RecyclerView*. Для отображения списков данных в *ListView* и *RecyclerView* используется адаптера (*Adapter*), связывающий данные с визуальными элементами в списке и отвечающий за создание и наполнение каждой строки или карточки в списке. В адаптере происходит создание элементов списка и связывание данных с элементами интерфейса.

ListView позволяет отображать элементы в виде списка и использует адаптер для подгрузки данных. Несмотря на свою простоту и наличие встроенных адаптеров, он имеет ограничения по производительности и гибкости настройки, а также сложности с реализацией анимаций, что привело к созданию нового компонента *RecyclerView*. *RecyclerView* является более современным и гибким компонентом для работы со списками и сетками данных. Он использует эффективный механизм отображения списков за счет повторного использования элементов (*view recycling*), что делает его оптимизированным для работы с большими наборами данных. *RecyclerView* позволяет настраивать расположение элементов с помощью *LayoutManager* и поддерживает различные типы адаптеров и обработку анимаций.

Компонентом, который позволяет размещать вид с прокруткой внутри другого вида с прокруткой является *NestedScrollView*. Внутри прокрутки может быть только один дочерний элемент. Поэтому для сложного макета необходимо использовать контейнер, внутри которого размещается другие компоненты.

1 ОСНОВНЫЕ СПОСОБЫ ОТОБРАЖЕНИЯ СПИСКОВ

Перед изучением возможностей отображения списков необходимо узнать про *LayoutInflater*. *LayoutInflater* используется для преобразования XML-файлов разметки в объекты *View*, которые возможно использовать в коде для создания интерфейсов. Основное предназначение *LayoutInflater* заключается в том, чтобы

динамически загружать иерархию элементов интерфейса во время выполнения программы. Основным методом предназначен для **инфляции макета (layout inflation)** *inflate*: преобразовании XML-файлов разметки в объекты View.

Существует несколько реализаций данного метода с различными параметрами, но все они используют друг друга и результат их выполнения один – View.

На слайде представлена одна из реализаций – `public View inflate (int resource, ViewGroup root, boolean attachToRoot)`. Аргументами метода является:

- *resource* - ID layout-файла, который будет использован для создания View. Например – (R.layout.main)

- *root* – родительский ViewGroup-элемент для создаваемого View. LayoutParams от этого ViewGroup присваиваются создаваемому View.

- *attachToRoot* – присоединять ли создаваемый View к root. Если true, то root становится родителем создаваемого View. Т.е. это равносильно команде `root.addView(View)`. Если false – то создаваемый View просто получает LayoutParams от root, но его дочерним элементом не становится.

Далее представлен пример реализации:

```
LayoutInflater inflater = LayoutInflater.from(this);  
View view = inflater.inflate(R.layout.info_layout, null);
```

1.1 ScrollView

Задание. Создать новый модуль. В меню «File | New | New Module | Phone & Tablet Module | Empty Views Activity». Название модуля «ScrollViewApp». Необходимо отобразить *геометрическую прогрессию со знаменателем 2 до 100 элемента*. Приложить скрин экрана в проект.

ScrollView является контейнером для размещения и отображения данных, которые не умещаются на экране. Он предоставляет возможность прокручивать содержимое вертикально или горизонтально. Основным предназначением является обеспечение прокрутки элементов интерфейса, таких как текст, изображения, кнопки, когда их высота или ширина превышает доступное пространство экрана. ScrollView может содержать только один прямой дочерний элемент, это означает, что внутри него размещается некоторый контейнер для элементов (например LinearLayout). ScrollView по умолчанию обеспечивает вертикальную прокрутку, для реализации горизонтальной прокрутки используется компонент HorizontalScrollView.

Для реализации *ScrollView* изначально требуется подготовить разметку элемента списка, пример структуры файла *item.xml* указан ниже.

```

<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:srcCompat="@drawable/ic_launcher_foreground" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="32dp"
        android:text="TextView"
        app:layout_constraintStart_toEndOf="@+id/imageView"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

После этого следует в файл с разметкой экрана добавить элемент `ScrollView`, внутри разместить элемент, который будет являться контейнером для списка. Данный способ целесообразно использовать если элементы в списке разные и их немного (максимум допустимая высота - несколько экранов, если больше, то стоит задуматься о другом решении).

```

<ScrollView
    android:id="@+id/scrollView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent">

    <LinearLayout
        android:id="@+id/wrapper"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical" />
</ScrollView>

```

В файле `activity` требуется с помощью *LayoutInflater* создать представление элемента списка из файла разметки, установить его значения и добавить его в контейнер.

```

LinearLayout wrapper = findViewById(R.id.wrapper);
for (int i=0; i < 100; i++){
    View view = getLayoutInflater().inflate(R.layout.item, null, false);
    TextView text = (TextView)view.findViewById(R.id.textView);
    text.setText(String.format("Элемент %d", i));
    wrapper.addView(view);
}

```

С помощью методов `scrollTo()` и `scrollBy()` возможно программно управлять прокруткой списка, например, прокручивать содержимое в нужную позицию.

```
public void scrollTo(int x, int y)
```

- `x` - позиция для прокрутки по оси X
- `y` - позиция для прокрутки по оси Y

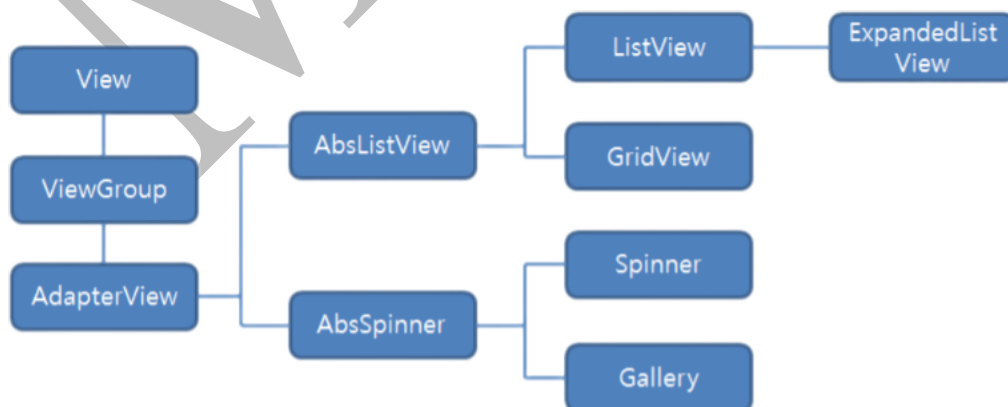
1.2 ListView

Задание. Создать новый модуль. В меню «File | New | New Module | Phone & Tablet Module | Empty Views Activity». Название модуля «*ListViewApp*». Создать список авторов и книг, которые Вы планируете прочитать в ближайшие 30 лет (более 30). Приложить скрин экрана в проект.

Следующим рассматриваемым компонентом является *ListView*. Предназначен для отображения небольших списков с одинаковыми элементами. Каждый элемент списка представляет собой отдельный элемент интерфейса, который может быть любым UI-компонентом (например, текст, изображение, кнопки).

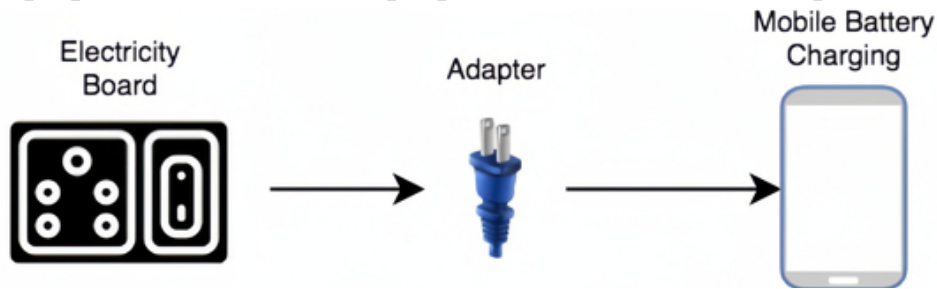
ListView поддерживает прокрутку, что позволяет отображать большое количество элементов в одном списке. Компонент не хранит данные внутри себя, для связывания данных с элементами списка используются адаптеры, что позволяет динамически загружать и отображать данные.

Android SDK предоставляет широкий набор элементов, к которым относятся списки. Все они являются наследниками класса *android.widget.AdapterView*.



К таким наследникам относят такие компоненты как *ListView*, *GridView*, *Spinner*, *Gallery*. Они могут выступать контейнерами для других элементов

управления. При работе со списками разработчик имеет дело с тремя компонентами.



Во-первых, это визуальный элемент или виджет, который на экране представляет список (*ListView*, *GridView*) и отображает данные (напр. *мобильное устройство отображает статус заряда*). Во-вторых, это источник данных – массив, объект *ArrayList*, база данных и т.д., в котором находятся сами отображаемые данные (напр. *электрическая сеть*). Третий компонент, это адаптер – специальный компонент, который связывает источник данных с виджетом списка.

Адаптер является мостом между компонентом пользовательского интерфейса и источником данных, который помогает заполнять данные в компоненте интерфейса. Адаптер расширяет абстрактный класс *AdapterView* (*ViewGroup*, который отображает элементы, загруженные в адаптер). Возможно создавать адаптер, либо использовать один из нескольких встроенных. К реализованным адаптерам относятся *ArrayAdapter* и *SimpleAdapter*.

Класс *ArrayAdapter* представляет собой простейший адаптер, который связывает массив данных с набором элементов *TextView*, из которых, к примеру, может состоять *ListView*. То есть в данном случае источником данных выступает массив объектов. *ArrayAdapter* вызывает у каждого объекта метод *toString()* для приведения к строковому виду и полученную строку устанавливает в элемент *TextView*.

```
public ArrayAdapter(@NonNull Context context, @LayoutRes int resource, @NonNull T[] objects)
```

- *context* - текущий контекст
- *resource* - ID ресурса для файла макета, содержащего *TextView*, для использования при создании экземпляров представлений
- *objects* - объекты для представления в *ListView*

Если список длинный, то необходимо обязательно реализовывать паттерн *ViewHolder*. Суть шаблона *ViewHolder* заключается в том, что он «запоминает» различные view-элементы для каждого списка, сокращая тем самым время и ресурсы, которые бы потребовались для постоянного создания и поиска этих элементов. Однако, на больших списках задержки в работе *ListView* неизбежны во многих случаях.

Создадим новый модуль *listViewApp*. В первую очередь разместим в файле разметки компонент *listView*

```
<ListView
    android:id="@+id/country_list_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Далее необходимо создать массив строк, содержащий некоторых стран БРИКС.

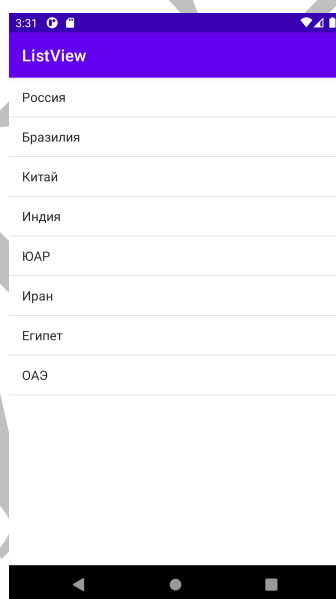
```
private String[] countries = { "Россия", "Бразилия", "Китай", "Индия", "ЮАР",
    "Иран", "Египет", "ОАЭ"};
```

Для отображения списка будет использоваться простейший адаптер *ArrayAdapter*. Создать адаптер данного типа возможно следующим образом:

```
ArrayAdapter adapter = new ArrayAdapter(this,
    android.R.layout.simple_list_item_2, android.R.id.text1, countries) {
```

В параметрах указывается контекст, XML-разметка элемента списка и массив данных. Контекстом может быть сама активность (*this*), под разметкой подразумевается компонент, в котором выводится текст, а данными является подготовленный массив, все элементы которого по очереди устанавливаются в указанную разметку. На последнем этапе необходимо списку передать адаптер.

```
countriesList.setAdapter(adapter);
```



Представим, что необходимо выводить данные не в одно текстовое поле, а в два. Стандартная разметка для списка с одним *TextView* не подходит. В стандартном наборе Android SDK имеются *simple_list_item_2* (состоит из 2 *TextView*, расположенных друг под другом), *simple_list_item_checked* (поддерживает возможность выбора) и др. Придётся самостоятельно наполнять нужную разметку данными.

Далее представлен пример переопределения метода *getView()*. Метод *getView()* принимает в качестве параметров позицию элемента, в котором будет выводиться информация, компонент для отображения данных (или *null*), а также родительский объект *ViewGroup*, в котором указанный компонент поместится. Вызов метода *getItem()* вернёт значение из исходного массива по указанному индексу. В результате

метод `getView()` должен вернуть экземпляр компонента, наполненный данными.

```
ArrayAdapter adapter = new ArrayAdapter(this,
    android.R.layout.simple_list_item_2, android.R.id.text1, countries) {
    @Override
    public View getView(int position, View convertView, ViewGroup parent)
    {
        View view = super.getView(position, convertView, parent);
        TextView text1 = view.findViewById(android.R.id.text1);
        TextView text2 = view.findViewById(android.R.id.text2);

        text2.setText(getItem(position).toString());
        text1.setText(String.valueOf(position+1));
        return view;
    }
};

countriesList.setAdapter(adapter);
}
```

1.3 RecyclerView

Задание. Создать новый модуль. В меню «File | New | New Module | Phone & Tablet Module | Empty Views Activity». Название модуля «*ListViewApp*». Создать список исторических событий с кратким описанием и изображением. Приложить скрин экрана в проект.

RecyclerView является мощным и гибким инструментом для отображения больших наборов данных в виде прокручиваемого списка или сетки. Он пришел на смену *ListView* и *GridView* и предлагает более эффективное управление отображением больших объемов данных. *RecyclerView* оптимизирует работу с элементами списка за счет повторного использования виджетов (так называемая «рециркуляция» элементов), что повышает производительность при работе с большими наборами данных.

Основными элементами *RecyclerView* являются *Adapter*, *ViewHolder* и *LayoutManager*. *Adapter* работает как посредник между набором данных и *RecyclerView*, отвечая за создание представлений для каждого элемента списка. *ViewHolder* хранит ссылки на элементы пользовательского интерфейса и используется для повышения производительности. Это достигается за счет того, что каждый раз не производится поиск элементов интерфейса при перелистывании экрана. *LayoutManager* управляет расположением элементов в списке (в виде линейного списка, сетки или собственной разметки экрана). Механизм переиспользования (*view recycling*) позволяет повторно использовать уже созданные представления, выходящие за пределы экрана, для новых элементов списка, что снижает потребление памяти и повышает производительность. *RecyclerView* поддерживает добавление стилизаций между элементами списка (например,

разделителей) с помощью *ItemDecoration*, реализацию анимаций для добавления, удаления или перемещения элементов возможна с помощью *ItemAnimator*. *ItemTouchHelper* в Android помогает реализовывать жесты для работы с элементами списка, такие как перетаскивание (drag-and-drop) и смахивание (swipe-to-dismiss). Он используется для добавления интерактивности и улучшения UX в списках или сетках, созданных с помощью *RecyclerView*.

RecycledViewPool отвечает за управление пулами для повторного использования *ViewHolder* в *RecyclerView*. Основная задача *RecycledViewPool* — кэшировать экземпляры *ViewHolder*, чтобы они могли повторно использоваться разными *RecyclerView*, что особенно полезно, когда имеются несколько *RecyclerView* на одном экране. Когда элемент списка удаляется из видимой области в *RecyclerView*, его *ViewHolder* попадает в кэш для повторного использования. По умолчанию *RecyclerView* имеет свой внутренний пул для управления кэшированием, но *RecycledViewPool* позволяет создавать общий пул для нескольких *RecyclerView*.

Класс *ViewHolder* является вспомогательным и предназначен для хранения ссылки на компоненты интерфейса для одного элемента списка, чтобы избежать лишних вызовов метода *findViewById()* во время прокрутки списков и обеспечить повторное переиспользование представлений. Он служит для оптимизации работы *RecyclerView*, улучшая производительность при отображении больших списков.

ViewHolder должен наследоваться от класса *RecyclerView.ViewHolder*. Как правило этот класс располагают внутри адаптера. В классе адаптера нужно обязательно переопределить 3 метода:

onCreateViewHolder() - вызывается, когда *RecyclerView* нуждается в новом *ViewHolder*. Внутри данного метода создается новый объект *ViewHolder* на основе макета элемента списка (*item_layout.xml*)

onBindViewHolder() - вызывается для каждого элемента списка, который становится видимым. Здесь происходит привязка данных к *ViewHolder* (например, установка текста и изображения).

getItemCount() - возвращает общее количество элементов в списке.

onViewAttachedToWindow – вызывается, когда представление, созданное этим адаптером, было присоединено к окну отображения

onViewDetachedFromWindow – вызывается, когда представление, созданное адаптером, было отсоединено от окна отображения.

onViewRecycled - представление перерабатывается, когда *RecyclerView.LayoutManager* решает, что некоторые элементы больше не требуются. Это может быть связано с тем, что элемент выпал из видимости или удалён из кэша. Если к представлению элемента привязаны большие или дорогостоящие данные, например большие растровые изображения, это может быть хорошим местом для

высвобождения этих ресурсов. RecyclerView вызывает этот метод прямо перед очисткой внутренних данных ViewHolder и отправкой их в RecyclerViewPool.

Далее будет представлен пример реализации списка основных участников стран BRICS с использованием RecyclerView. На первом этапе необходимо реализовать отображение данных и создать модель данных для слоя представления, содержащую информацию о названии страны, название флага, по которому будет происходить поиск в ресурсах, и количество населения.

```
public class Country {
    private String countryName;
    private String flagName;
    private int population;

    public Country(String countryName, String flagName, int population) {
        this.countryName= countryName;
        this.flagName= flagName;
        this.population= population;
    }

    public int getPopulation() {
        return population;
    }

    public String getCountryName() {
        return countryName;
    }

    public String getFlagName() {
        return flagName;
    }

    @Override
    public String toString() {
        return this.countryName+" (Population: "+ this.population+")";
    }
}
```

Далее создается внешний вид элемента списка в файлах разметки (*country_item_view.xml*).



Рисунок 1 - Внешний вид файла разметки *country_item_view.xml*

Элемент списка реализован на контейнерах *CardView* и *ConstraintLayout* и содержит компоненты *ImageView* и два текстовых поля.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <ImageView
            android:id="@+id/imageView"
            .../>

        <TextView
            android:id="@+id/textViewCountryName"
            .../>

        <TextView
            android:id="@+id/textViewPopulation"
            ... />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.cardview.widget.CardView>

```

В *activity_main.xml* также необходимо внести изменения в разметку и разместить компонент *RecyclerView*.

```

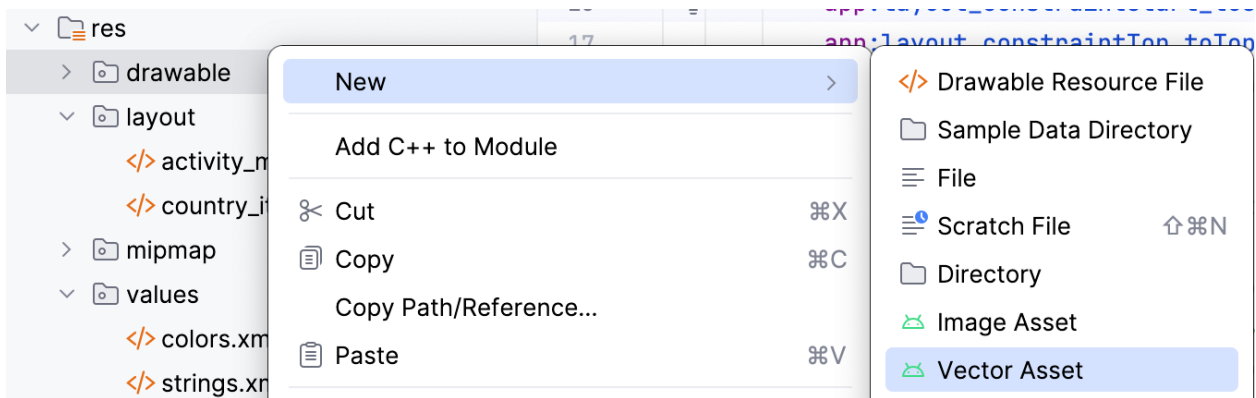
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Так как имеется требование по отображению флагов государств, то необходимо их импортировать в проект. Наиболее оптимальный способ использования такого рода изображений является работа с векторными изображениями. На ресурсе [Wikimedia.org](https://commons.wikimedia.org/) возможно скачать изображения флагов в формате *.svg*. Для импортирования в проект изображений необходимо на директории *drawable* вызвать контекстное меню->New->Vector Asset и указать путь до файла.



На втором этапе после формирования модели данных и файлов разметки необходимо создать *ViewHolder*. Данный класс отвечает за связывания модели данных и элементов пользовательского интерфейса, которые будут отображаться в одном элементе списка. Создается класс *CountryViewHolder*.

```
public class CountryViewHolder extends RecyclerView.ViewHolder {
    private ImageView flagView;
    private TextView countryNameView;
    private TextView populationView;
    public CountryViewHolder(@NonNull View itemView) {
        super(itemView);
        this.flagView = itemView.findViewById(R.id.imageView);
        this.countryNameView =
itemView.findViewById(R.id.textViewCountryName);
        this.populationView = itemView.findViewById(R.id.textViewPopulation);
    }

    public ImageView getFlagView() {
        return flagView;
    }

    public TextView getCountryNameView() {
        return countryNameView;
    }

    public TextView getPopulationView() {
        return populationView;
    }
}
```

На третьем этапе необходимо создать Adapter для связывания данных с ViewHolder'ами. Для реализации собственного адаптера требуется переопределить 3 метода класса RecyclerView.Adapter. В конструктор класса передается список объектов *Country*. Метод *onCreateViewHodler* вызывается при создании нового объекта ViewHolder и создает новое представление на основе предоставленной разметки в файле *country_item_view*. В методе *getItemCount()* возвращается количество элементов списка, сообщает RecyclerView, сколько элементов данных необходимо отобразить.

```

public class CountryRecyclerViewAdapter extends
RecyclerView.Adapter<CountryViewHolder> {
    private List<Country> countries;
    private Context context;

    public CountryRecyclerViewAdapter(List<Country> countries) {
        this.countries = countries;
    }

    @Override
    public CountryViewHolder onCreateViewHolder(final ViewGroup parent, int
viewType) {
        // Inflate view from recyclerview_item_layout.xml
        context = parent.getContext();
        View recyclerViewItem =
LayoutInflater.from(context).inflate(R.layout.country_item_view, parent,
false);

        return new CountryViewHolder(recyclerViewItem);
    }

    @Override
    public void onBindViewHolder(CountryViewHolder holder, int position) {
        // Cet country in countries via position
    }

    @Override
    public int getItemCount() {
        return this.countries.size();
    }
}

```

Далее необходимо связать данные с конкретным представлением. Метод *onBindViewHolder* вызывается для каждого элемента списка, когда данные должны быть обновлены, передавая данные на основе позиции элемента в списке.

```

@Override
public void onBindViewHolder(CountryViewHolder holder, int position) {
    // Cet country in countries via position
    Country country = this.countries.get(position);
    String pkgName = context.getPackageName();
    // Return 0 if not found.
    int resID = context.getResources().getIdentifier(country.getFlagName() ,
"drawable", pkgName);
    // Bind data to viewholder
    holder.getFlagView().setImageResource(resID);
    holder.getCountryNameView().setText(country.getCountryName());
    holder.getPopulationView().setText("Population: " +
country.getPopulation());
}

```

На завершающем этапе необходимо настроить RecyclerView в Activity и создать массив стран.

```

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        List<Country> countries = getListData();
        RecyclerView recyclerView = this.findViewById(R.id.recyclerView);
        recyclerView.setAdapter(new CountryRecyclerViewAdapter(countries));

        // RecyclerView scroll vertical
        LinearLayoutManager layoutManager = new LinearLayoutManager(this,
        LinearLayoutManager.VERTICAL, false);
        recyclerView.setLayoutManager(layoutManager);

    }

    private List<Country> getListData() {
        List<Country> list = new ArrayList<Country>();
        Country brazil = new Country("Brazil", "flag_of_brazil", 210000000);
        Country china = new Country("China", "flag_of_china", 1420000000);
        Country india = new Country("India", "flag_of_india", 1430000000);
        Country russia = new Country("Russia", "flag_of_russia", 146000000 );
        Country southAfrica = new Country("South Africa",
        "flag_of_south_africa", 64000000 );

        list.add(brazil);
        list.add(china);
        list.add(india);
        list.add(russia);
        list.add(southAfrica);

        return list;
    }
}

```

В результате на экране должен отобразиться список стран.



Далее возможно модифицировать внешний вид LayoutManger.

1. **LinearLayoutManager** отображает элементы в виде вертикального или горизонтального списка.

```
LinearLayoutManager layoutManager = new LinearLayoutManager(this,
LinearLayoutManager.HORIZONTAL, false);
recyclerView.setLayoutManager(layoutManager);
```

2. **GridLayoutManager** отображает элементы в виде сетки.

```
GridLayoutManager layoutManager = new GridLayoutManager(this, 2); //
recyclerView.setLayoutManager(layoutManager);
```

3. **StaggeredGridLayoutManager** отображает элементы с разной высотой, как в Pinterest.

```
StaggeredGridLayoutManager layoutManager = new StaggeredGridLayoutManager(2,
StaggeredGridLayoutManager.VERTICAL); // 2 столбца
recyclerView.setLayoutManager(layoutManager);
```

Стоит отметить, что имеется возможность добавления разделителей (например, линии между элементами), отступы или другие графические элемент.

```
DividerItemDecoration divider = new DividerItemDecoration(this,
layoutManager.getOrientation());
recyclerView.addItemDecoration(divider);
recyclerView.setLayoutManager(layoutManager);
```

Чтобы добавить обработчик кликов для элементов, возможно добавить обработчик в методе `onBindViewHolder`.

```
holder.itemView.setOnClickListener(v -> {
    // Обработка клика на элемент
    Toast.makeText(this.context, "Clicked: " + country.getCountryName(),
    Toast.LENGTH_LONG).show();
});
```

2 КОНТРОЛЬНОЕ ЗАДАНИЕ

В ходе выполнения практического задания № 1 были сформулированы use-case возможности проекта. Приложение должно удовлетворять следующим функциональным требованиям:

1. Авторизация в приложении.
2. Взаимодействия с каким-либо внешним сервисом, возвращающим данные о стоимости валюты, погоды и т.д. в виде json-формата. Возможно использовать сервисы типа <https://www.wiremock.io/>, <https://mockapi.io/>, <https://github.com/typicode/json-server>, <https://thecatapi.com/> и т.д.
3. Сохранение данных в БД.
4. Отображение списка каких-либо сущностей с изображениями.
5. Отображение страницы сущности.
6. Различные возможности у авторизованного и гостевого пользователя.
7. Использование обученной модели для распознавания изображений, аудио, маски и т.д. (нужно произвести анализ существующих обученных моделей TensorFlow Lite, которые желаете использовать в приложении).

Задание. Требуется открыть приложение из практического занятия №1. Ваша реализация и смысл приложения отличаются от представленного в практическом задании примера. Необходимо изменить приложение по аналогии с примером. В репозитории необходимо создать заглушка с набором данных (напр. которые планируете использовать при взаимодействии с API внешнего сервиса). Передать эти данные в слой представления с помощью LiveData и установить в RecyclerView.

Далее представлен пример работы с *RecyclerView* во *ViewModel*.

```
public class CountryViewModel extends ViewModel {
    private final MutableLiveData<List<Country>> items = new
MutableLiveData<>();

    public CountryViewModel() {
        // Инициализируем начальные данные
        List<Country> itemList = new ArrayList<>();
        itemList.add(new Country("Brazil", "flag_of_brazil", 210000000));
        itemList.add(new Country("China", "flag_of_china", 1420000000));
        itemList.add(new Country("India", "flag_of_india", 1430000000));
        itemList.add(new Country("Russia", "flag_of_russia", 1460000000));
        itemList.add(new Country("South Africa", "flag_of_south_africa",
64000000));
        items.setValue(itemList);
    }

    public LiveData<List<Country>> getItems() {
        return items;
    }

    // Метод для добавления нового элемента в список
    public void addItem(Country item) {
        List<Country> currentList = items.getValue();
        if (currentList != null) {
            currentList.add(item);
            items.setValue(currentList); // Обновляем LiveData
        }
    }
}
```

```

public class CountryAdapter extends
RecyclerView.Adapter<CountryAdapter.SimpleCountryViewHolder> {
    private List<Country> itemList = new ArrayList<>();

    public void setItems(List<Country> items) {
        this.itemList = items;
        notifyDataSetChanged();
    }

    @NonNull
    @Override
    public SimpleCountryViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
        View recyclerViewItem =
LayoutInflater.from(parent.getContext()).inflate(R.layout.country_item_view,
parent, false);
        return new SimpleCountryViewHolder(recyclerViewItem);
    }

    @Override
    public void onBindViewHolder(@NonNull SimpleCountryViewHolder holder, int
position) {
        holder.bind(itemList.get(position));
    }

    @Override
    public int getItemCount() {
        return itemList.size();
    }

    public class SimpleCountryViewHolder extends RecyclerView.ViewHolder {
        private TextView countryNameView;
        public SimpleCountryViewHolder(@NonNull View itemView) {
            super(itemView);
            this.countryNameView =
itemView.findViewById(R.id.textViewCountryName);
        }

        public void bind(Country item) {
            countryNameView.setText(item.getCountryName());
        }
    }
}

```

Main activity

```
public class MainActivity extends AppCompatActivity {
    private CountryViewModel itemViewModel;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // List<Country> countries = getListData();

        RecyclerView recyclerView = this.findViewById(R.id.recyclerView);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));

        CountryAdapter itemAdapter = new CountryAdapter();
        recyclerView.setAdapter(itemAdapter);

        itemViewModel = new
        ViewModelProvider(this).get(CountryViewModel.class);
        // Добавляем наблюдатель для LiveData
        itemViewModel.getItems().observe(this, items -> {
            itemAdapter.setItems(items); // Обновляем данные в адаптере
        });
    }
}
```